

Figure 4: Weird data points

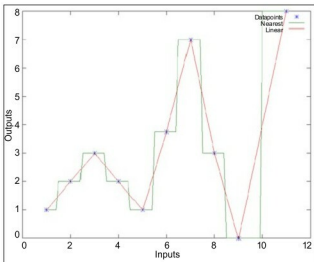


Figure 5: Nearest and linear interpolations of the data points

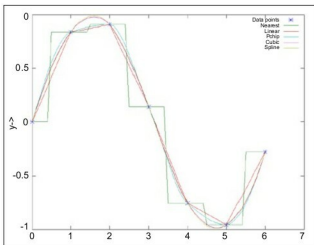


Figure 6: All types of interpolations for sinusoidal points

```
octave:9> std_dev = sqrt(mean((y - polyval(p, x)).^2))
std_dev = 0.26873
```

```
octave:10>
```

Figure 3 shows the data points and fitted quadratic curve. All the values above can be interpreted in the similar way as earlier. And we note that the standard deviation has come down further. In fact, as we increase the polynomial order further and further, we will note that the standard deviation keeps on decreasing, till the order is the same as the number of data points - 1, after which it remains 0. But, we may not like to do this, as our observed data points are typically not perfect, and we are not interested in getting the standard deviation zero but in getting a fit to reflect more appropriately the relation between the system's input and output.

Interpolation

Now, let's assume we have the following weird data points, listed as (x, y) pairs: (1, 1), (2, 2), (3, 3), (4, 2), (5, 1), (6, 3.75), (7, 7), (8, 3), (9, 0), (11, 8). Figure 4 shows how weird they look.

Trying to fit a polynomial curve would be quite meaningless. But, still, the points seem to have some correlation between smaller ranges of input, say between 1 and 3, 3 and 5, 5 and 7, and so on. So, this is best suited for us to do interpolation to find the values in between, say at 1.5, 3.8, 10, ... Now interpolation is all about curve fitting between two neighbouring data points and then calculating the value at any intermediate point. The standard set of techniques used for this 'piece-wise curve fitting' is:

- **nearest**: Return the nearest data point (actually no curve fitting)
- **linear**: Linear/straight line fitting between the two neighbouring data points
- **pchip**: Piece-wise cubic (order 3) hermite polynomial fitting
- **cubic**: Cubic (order 3) polynomial fitted between four nearest neighbouring data points
- **spline**: Cubic (order 3) spline fitting with smooth first and second derivatives throughout the curve

All this may sound like jargon but don't worry about that.

Let's just try out some examples to drive home the point.

Here's the code to show the nearest and linear interpolation for the above data:

```
octave:1> x = [1 2 3 4 5 6 7 8 9 11];
octave:2> y = [1 2 3 2 1 3.75 7 3 0 8];
octave:3> x1 = 1:0.1:11;
octave:4> y_n = interp1(x, y, x1, "nearest");
octave:5> y_l = interp1(x, y, x1, "linear");
octave:6> plot(x, y, "x", x1, y_n, "-.", x1, y_l, "-");
octave:7> xlabel("Inputs");
octave:8> ylabel("Outputs");
octave:9> legend("Data points", "Nearest", "Linear");
octave:10>
```

Continued on page 97...